

Docker Compose Command Guide

Detailed commands, examples, descriptions, and practical Docker Compose workflows

Best for: Beginners and trainers who want to understand Docker Compose from scratch, especially for Python, Streamlit, MySQL, and multi-container projects.

Compose version: Modern Docker uses the command `docker compose` with a space. Older installations may use `docker-compose` with a hyphen. Prefer `docker compose` when Docker Desktop or Docker Engine has the Compose plugin installed.

Table of Contents

- 1. What is Docker Compose?
- 2. Docker Image vs Container vs Compose
- 3. Compose file structure
- 4. Full example: Streamlit + MySQL
- 5. Basic command pattern
- 6. Important global options
- 7. Core Docker Compose commands
- 8. Lifecycle commands
- 9. Build and image commands
- 10. Debugging commands
- 11. Volume and network commands
- 12. Environment variables and profiles
- 13. Common workflows
- 14. Troubleshooting
- 15. Cheat sheet

1. What is Docker Compose?

Docker Compose is a tool used to define and run multiple Docker containers together. Instead of running many long docker run commands manually, you write one YAML file, usually named compose.yaml or docker-compose.yml, and start the full application using one command.

For example, a real application may need one Python Streamlit container, one MySQL container, one Redis container, and maybe one monitoring container. Docker Compose helps you start, stop, rebuild, connect, and manage all of them as one project.

```
docker compose up
```

This single command reads your Compose file, creates the required network and volumes, builds or pulls the images, and starts all services.

Simple meaning: Docker Compose is like a project manager for containers. Docker runs one container. Docker Compose runs the full multi-container application.

2. Docker Image vs Container vs Docker Compose

Term	Meaning	Example
Docker image	A read-only package containing application code, runtime, libraries, and dependencies.	python:3.11, mysql:8.0, yourname/myapp:v1
Docker container	A running instance of an image. It has its own process, filesystem layer, networking, and environment.	streamlit_container running from your Streamlit image
Docker Compose	A YAML-based method to run multiple containers as one application stack.	streamlit_app + mysql_db + named volume + network

3. Compose file structure

A Compose file is written in YAML. YAML is indentation-sensitive, so spaces matter. Do not use tabs.

```
services:
  mysql_db:
    image: mysql:8.0
    container_name: mysql_container
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: userdb
    ports:
      - "3307:3306"
    volumes:
      - mysql_data:/var/lib/mysql

  streamlit_app:
```

```

build: .
container_name: streamlit_container
restart: always
ports:
  - "8501:8501"
depends_on:
  - mysql_db

volumes:
  mysql_data:

```

3.1 Important Compose file sections

Section / Key	Description	Example
services	Defines the containers in your application.	mysql_db, streamlit_app
image	Uses an existing image from Docker Hub or local image cache.	image: mysql:8.0
build	Builds an image using a Dockerfile from the given path.	build: .
container_name	Sets a fixed name for the created container.	container_name: mysql_container
restart	Restart policy for the container.	restart: always
environment	Environment variables passed inside the container.	MYSQL_ROOT_PASSWORD: root123
ports	Maps host machine ports to container ports.	"8501:8501"
volumes	Stores persistent data or mounts files/folders.	mysql_data:/var/lib/mysql
depends_on	Starts one service before another service.	streamlit_app depends on mysql_db
networks	Defines custom container networks.	backend_network
profiles	Starts optional services only when profile is enabled.	profiles: [monitoring]

4. Full example: Streamlit + MySQL

Folder structure:

```

my_compose_project/
  compose.yaml
  Dockerfile
  requirements.txt
  app/
    app.py

```

Dockerfile:

```

FROM python:3.11
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt

```

```
COPY app/ .
EXPOSE 8501
CMD ["streamlit", "run", "app.py", "--server.address=0.0.0.0"]
```

requirements.txt:

```
streamlit
mysql-connector-python
```

compose.yaml:

```
services:
  mysql_db:
    image: mysql:8.0
    container_name: mysql_container
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: root123
      MYSQL_DATABASE: userdb
    ports:
      - "3307:3306"
    volumes:
      - mysql_data:/var/lib/mysql

  streamlit_app:
    build: .
    container_name: streamlit_container
    restart: always
    ports:
      - "8501:8501"
    depends_on:
      - mysql_db

volumes:
  mysql_data:
```

Important: Inside the Streamlit container, connect to MySQL using host mysql_db and port 3306. From your Windows host machine, connect using localhost and port 3307.

5. Basic Docker Compose command pattern

```
docker compose [OPTIONS] COMMAND [ARGS]
```

Examples:

```
docker compose up
docker compose up -d
docker compose down
docker compose ps
docker compose logs -f
```

6. Important global options

-f, --file

Use a specific Compose file instead of the default compose.yaml or docker-compose.yml.

```
docker compose -f compose.dev.yaml up -d
```

-p, --project-name

Set the Compose project name. This affects generated network, volume, and container names.

```
docker compose -p trainingapp up -d
```

--profile

Enable services that are assigned to a specific profile.

```
docker compose --profile monitoring up -d
```

--env-file

Use a specific environment variable file.

```
docker compose --env-file .env.dev up -d
```

--project-directory

Set the working directory for Compose project files.

```
docker compose --project-directory ./myapp up -d
```

--ansi

Control colored output. Useful in CI logs.

```
docker compose --ansi never up
```

--progress

Control build progress output.

```
docker compose --progress plain build
```

7. Core Docker Compose commands

docker compose version

Show Docker Compose version information.

Example:

```
docker compose version
```

Description: Use this first to confirm that Docker Compose is installed. Modern output usually shows Docker Compose version v2.x.

docker compose help

Show help for Docker Compose or a specific command.

Example:

```
docker compose help
docker compose up --help
```

Description: Useful when you forget options for a command. The second command shows only help for up.

docker compose config

Validate and display the final Compose configuration after variable substitution and file merging.

Example:

```
docker compose config
```

Description: Use this before running up when you want to confirm indentation, variables, ports, volumes, and merged files are correct.

docker compose up

Create and start all services defined in the Compose file.

Example:

```
docker compose up
```

Description: Runs in foreground. You can see logs directly. Press Ctrl+C to stop foreground output.

docker compose up -d

Start services in detached/background mode.

Example:

```
docker compose up -d
```

Description: Most common production-style command. Containers continue running after the terminal is closed.

docker compose up --build

Build images before starting containers.

Example:

```
docker compose up --build
```

Description: Use this after changing Dockerfile, requirements.txt, or app code copied into the image.

docker compose up --force-recreate

Recreate containers even if their configuration did not change.

Example:

```
docker compose up -d --force-recreate
```

Description: Useful when you want a clean container recreation but want to keep named volumes.

docker compose up --no-deps

Start or recreate only selected service without starting dependencies.

Example:

```
docker compose up -d --no-deps streamlit_app
```

Description: Useful when only application code changed and you do not want to touch the database container.

docker compose down

Stop and remove containers, default networks, and related Compose resources.

Example:

```
docker compose down
```

Description: Does not delete named volumes by default. Database data in `mysql_data` remains safe unless you use `-v`.

docker compose down -v

Stop containers and remove named volumes declared in the Compose file.

Example:

```
docker compose down -v
```

Description: Danger: this deletes persistent database data stored in named volumes.

docker compose down --rmi local

Stop containers and remove local images built by Compose.

Example:

```
docker compose down --rmi local
```

Description: Useful when you want to clean images built locally by the project.

docker compose start

Start existing stopped containers without recreating them.

Example:

```
docker compose start
```

Description: Use this when containers already exist and you only stopped them earlier.

docker compose stop

Stop running containers without removing them.

Example:

```
docker compose stop
```


Description: This preserves containers, networks, and volumes. You can start again using docker compose start.

docker compose restart

Restart services.

Example:

```
docker compose restart
docker compose restart streamlit_app
```

Description: Useful after changing environment outside the container or when a service is stuck.

docker compose pause

Pause running containers.

Example:

```
docker compose pause
```

Description: Freezes processes inside containers. Not commonly used in training or production.

docker compose unpause

Resume paused containers.

Example:

```
docker compose unpause
```

Description: Use after pause.

docker compose kill

Force stop containers by sending a signal.

Example:

```
docker compose kill
docker compose kill -s SIGKILL mysql_db
```

Description: Use only when normal stop does not work. It is abrupt and may not allow graceful shutdown.

docker compose ps

List containers in the Compose project.

Example:

```
docker compose ps
```

Description: Shows service name, container name, command, status, and published ports.

docker compose ls

List Compose projects.

Example:

```
docker compose ls
```

Description: Useful when you have multiple Compose projects running on the same system.

docker compose logs

Show service logs.

Example:

```
docker compose logs
```

Description: Displays combined logs from all services.

docker compose logs -f

Follow live logs.

Example:

```
docker compose logs -f streamlit_app
```

Description: Useful for debugging application errors while the app is running.

docker compose logs --tail

Show only the latest log lines.

Example:

```
docker compose logs --tail=100 mysql_db
```

Description: Useful when logs are very large.

docker compose exec

Run a command inside an already running container.

Example:

```
docker compose exec mysql_db bash  
docker compose exec streamlit_app python --version
```

Description: Best command for debugging a running service.

docker compose run

Run a one-off command in a new container for a service.

Example:

```
docker compose run --rm streamlit_app python --version
```

Description: Creates a temporary container. Use --rm to delete it after command completes.

docker compose build

Build or rebuild images for services that use build.

Example:

```
docker compose build
docker compose build streamlit_app
```

Description: Use when you changed Dockerfile or dependencies. Does not start containers by itself.

docker compose build --no-cache

Build image without using Docker build cache.

Example:

```
docker compose build --no-cache streamlit_app
```

Description: Useful when package installation or cached layers cause issues.

docker compose pull

Pull service images from a registry such as Docker Hub.

Example:

```
docker compose pull
docker compose pull mysql_db
```

Description: Used for services with image: key, like mysql:8.0.

docker compose push

Push service images to a registry.

Example:

```
docker compose push
```

Description: Works when service images have proper image names such as dirac2/lift_zero:v1 and you are logged in.

docker compose images

List images used by created containers.

Example:

```
docker compose images
```

Description: Helps confirm which image ID and tag each service is using.

docker compose create

Create containers without starting them.

Example:

```
docker compose create
```

Description: Useful for checking creation separately from startup, but less common than up.

docker compose rm

Remove stopped service containers.

Example:

```
docker compose rm
docker compose rm -f streamlit_app
```

Description: Use after docker compose stop if you want to remove containers without removing volumes.

docker compose cp

Copy files between host and a service container.

Example:

```
docker compose cp streamlit_app:/app/app.py ./app_copy.py
docker compose cp ./local.txt streamlit_app:/app/local.txt
```

Description: Useful for debugging or exporting files from a container.

docker compose port

Show public port mapping for a service port.

Example:

```
docker compose port mysql_db 3306
```

Description: For mysql_db, it may show 0.0.0.0:3307 when mapped as 3307:3306.

docker compose top

Display running processes in service containers.

Example:

```
docker compose top
```

Description: Useful for checking what process is running inside containers.

docker compose stats

Show live CPU, memory, network, and IO usage.

Example:

```
docker compose stats
```

Description: Useful for monitoring resource usage during testing.

docker compose events

Stream container events for the Compose project.

Example:

```
docker compose events
```

Description: Advanced debugging command to see create, start, stop, die events.

docker compose wait

Wait until containers stop and print their exit codes.

Example:

```
docker compose wait
```

Description: Useful in CI/CD or batch jobs where containers are expected to finish.

docker compose watch

Watch source files and automatically rebuild/restart/sync based on develop rules.

Example:

```
docker compose watch
```

Description: Useful in development with compatible Compose setup. It is for live development workflows.

8. Practical lifecycle workflows

First time run

```
docker compose config  
docker compose up --build
```

Validate the Compose file, build images, and run in foreground so you can see logs.

Run in background

```
docker compose up -d  
docker compose ps
```

Start the application in background and verify container status.

See logs while app is running

```
docker compose logs -f streamlit_app
```

Follow live logs only for the Streamlit service.

Open shell inside app container

```
docker compose exec streamlit_app bash
```

Enter the already running app container for debugging.

Open MySQL shell

```
docker compose exec mysql_db mysql -uroot -proot123 userdb
```

Connect to MySQL from inside the MySQL service container.

Rebuild after code changes

```
docker compose up -d --build streamlit_app
```

Rebuild and restart only the Streamlit service.

Stop without deleting data

```
docker compose stop
```

Stops containers. Keeps containers and volumes.

Remove containers but keep database volume

```
docker compose down
```

Removes containers and default network. Named volume mysql_data remains.

Delete everything including database data

```
docker compose down -v
```

Removes named volumes too. Use carefully.

Clean stopped containers

```
docker compose rm -f
```

Removes stopped Compose containers.

9. Docker Hub workflow with Docker Compose

To push a Compose-built image to Docker Hub, the service should have an image name. Example:

```
services:
  streamlit_app:
    build: .
    image: dirac2/lift_zero:v1
    container_name: streamlit_container
    ports:
      - "8501:8501"
```

Commands:

```
docker login
docker compose build streamlit_app
docker compose push streamlit_app
```

Alternative using normal Docker commands:

```
docker build -t dirac2/lift_zero:v1 .
docker push dirac2/lift_zero:v1
```

Important: docker compose push pushes images defined with image:. If a service has only build: and no image:, Compose does not know the Docker Hub repository name to push.

10. Ports, volumes, and networks in detail

10.1 Ports

```
ports:
  - "8501:8501"
  - "3307:3306"
```

Format is HOST_PORT:CONTAINER_PORT. For Streamlit, 8501:8501 means open localhost:8501 on your machine and forward traffic to port 8501 inside the container. For MySQL, 3307:3306 means connect from Windows using localhost:3307, but inside Docker network other containers use mysql_db:3306.

10.2 Named volumes

```
services:
  mysql_db:
    volumes:
      - mysql_data:/var/lib/mysql

volumes:
  mysql_data:
```

mysql_data is a named Docker volume. /var/lib/mysql is where MySQL stores database files inside the container. This mapping preserves database data even when the MySQL container is removed.

10.3 Bind mounts

```
services:
  streamlit_app:
    volumes:
      - ./app:/app
```

A bind mount maps a local folder into the container. This is useful during development because code changes on your host machine can be visible inside the container.

10.4 Networks

```
services:
  mysql_db:
    networks:
      - backend
  streamlit_app:
    networks:
      - backend

networks:
  backend:
```

Compose automatically creates a default network even if you do not define networks. Services can reach each other using service names like mysql_db. Custom networks are useful when separating frontend, backend, and database layers.

11. Environment variables and profiles

11.1 .env file

Create a .env file:

```
MYSQL_ROOT_PASSWORD=root123
MYSQL_DATABASE=userdb
APP_PORT=8501
```

Use variables in compose.yaml:

```
services:
  mysql_db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}

  streamlit_app:
    build: .
    ports:
      - "${APP_PORT}:8501"
```

Run:

```
docker compose up -d
```

11.2 Profiles

```
services:
  streamlit_app:
    build: .

  prometheus:
    image: prom/prometheus
    profiles:
      - monitoring
```

Without profile, only normal services start:

```
docker compose up -d
```

With monitoring profile:

```
docker compose --profile monitoring up -d
```

12. Common troubleshooting

Problem	Likely reason	Solution
Port already allocated	Another application or container is already using the host port.	Change the host port, for example from "8501:8501" to "8502:8501", or stop the other container.

My app cannot connect to MySQL	The app is using localhost inside the container.	Use mysql_db as host and 3306 as port from another container. Use localhost:3307 only from the host machine.
Data disappeared after down -v	The -v option removed named volumes.	Avoid down -v when you want to keep database data. Use docker compose down instead.
Code change not reflected	The image was not rebuilt or code is copied at build time.	Run docker compose up -d --build or use a bind mount during development.
Compose file indentation error	YAML indentation is wrong or tabs were used.	Use spaces only. Run docker compose config to validate.
Old container name conflict	A container with the same container_name already exists.	Remove old container using docker rm -f container_name or avoid fixed container_name.

13. Docker Compose cheat sheet

Task	Command
Validate file	docker compose config
Start foreground	docker compose up
Start background	docker compose up -d
Build and start	docker compose up --build
Stop services	docker compose stop
Start stopped services	docker compose start
Restart services	docker compose restart
Remove containers/network	docker compose down
Remove containers/network/volumes	docker compose down -v
List containers	docker compose ps
Show logs	docker compose logs
Follow logs	docker compose logs -f
Enter running container	docker compose exec service_name bash
Run one-off command	docker compose run --rm service_name command
Build image	docker compose build
Pull images	docker compose pull
Push images	docker compose push
Show images	docker compose images
Copy file	docker compose cp service:path localpath
Show resource usage	docker compose stats
Show processes	docker compose top

14. Recommended learning path

1. First understand one container using docker run.
2. Then create one Compose file with one service.
3. Next add MySQL with a named volume.
4. Then add a Python or Streamlit application service.
5. Practice logs, exec, build, up, down, and volume handling.
6. Finally learn profiles, environment files, and Docker Hub push workflows.